

Sitzer –

Specyfikacja wymagań i plan testów

Spis treści

1. Wprowadzenie	3
2. Cele biznesowe	3
3. Wymagania funkcjonalne	4
4. Technologie i architektura	7
5. Plan testów.....	8
6. Strategia testowania.....	8
7. Środowisko testowe.....	8
8. Scenariusz testów	9
9. Defekty i analiza błędów w modułach aplikacji	10

1. Wprowadzenie

- Cel projektu

Aplikacja została stworzona w celu praktycznego wykorzystania wiedzy z zakresu tworzenia aplikacji mobilnych oraz testowania oprogramowania.

- Zakres systemu

Aplikacja z grupy fitness & health, podejście offline-first.

- Krótki opis aplikacji

Sitzer to aplikacja stworzona dla ludzi chcących poprawić posturę oraz zredukować negatywne skutki długotrwałego siedzenia.

- Grupa docelowa użytkowników

Grupą docelową aplikacji są osoby, które prowadzą siedzący tryb życia, głównie pracowników biurowych, graczy komputerowych czy ludzi, których codzienne obowiązki i praca wymagają godzin w pozycji siedzącej.

2. Cele biznesowe

Aplikacja jest rozwiązaniem na problemy z systematyką wykonywania ćwiczeń czy samym zaczęciem wprowadzania w codzienne życie ćwiczeń posturalnych. Potencjalny użytkownik może za pomocą specjalnie przygotowanych planów wzmocnić czy rozciągnąć mięśnie które do tej pory sprawiały dyskomfort. Przystępna forma aplikacji mobilnej sprawia, że urządzenie jak smartfon może stać się narzędziem do wprowadzenia w życie codzienne zdrowych nawyków i zredukowania skutków długotrwałego siedzenia.

3. Wymagania funkcjonalne

- **Uwierzytelnianie użytkownika (Rejestracja / Logowanie / Wylogowanie)**

ID	Wymaganie	Priorytet
FR-Auth-01	System umożliwia rejestrację nowego użytkownika poprzez podanie imienia, adresu e-mail, hasła i potwierdzenia hasła.	Wysoki
FR-Auth-02	System waliduje, czy pole imienia nie jest puste przed rejestracją.	Wysoki
FR-Auth-03	System waliduje format adresu e-mail (musi zawierać @ i domenę).	Wysoki
FR-Auth-04	System wymaga, aby hasło miało minimum 6 znaków.	Wysoki
FR-Auth-05	System weryfikuje zgodność hasła z polem potwierdzenia hasła.	Wysoki
FR-Auth-06	System wyświetla komunikat błędu przy próbie rejestracji z istniejącym adresem e-mail.	Wysoki
FR-Auth-07	System umożliwia logowanie użytkownika za pomocą adresu e-mail i hasła.	Wysoki
FR-Auth-08	System wyświetla odpowiedni komunikat błędu przy podaniu błędnych danych logowania.	Wysoki
FR-Auth-09	Przycisk logowania jest aktywny wyłącznie, gdy oba pola (e-mail i hasło) są wypełnione.	Średni
FR-Auth-10	System przekierowuje zalogowanego użytkownika do ekranu głównego (HomeScreen).	Wysoki
FR-Auth-11	System umożliwia wylogowanie użytkownika z poziomu ekranu profilu.	Średni
FR-Auth-12	Dane użytkownika są zapisywane lokalnie w bazie danych Room.	Wysoki

- **Ekran główny (HomeScreen)**

ID	Wymaganie	Priorytet
FR-HOME-01	Ekran główny wyświetla aktualny dzienny plan ćwiczeń przypisany do użytkownika.	Wysoki
FR-HOME-02	Ekran prezentuje postępy treningowe w postaci kart statystyk.	Wysoki
FR-HOME-03	Ekran wyświetla spersonalizowane powitanie z imieniem zalogowanego użytkownika.	Średni
FR-HOME-04	Ekran umożliwia nawigację do ekranu wyboru planów (PlanSelectScreen).	Wysoki
FR-HOME-05	Ekran główny jest dostępny wyłącznie dla zalogowanych użytkowników.	Wysoki
FR-HOME-06	Ekran umożliwia nawigację do centrum treningowego (WorkoutHubScreen).	Wysoki

- **Plany Treningowe (PlanSelectScreen)**

ID	Wymaganie	Priorytet
FR-PLAN-01	System udostępnia trzy poziomy planów treningowych: Beginner, Intermediate, Advanced.	Wysoki

FR-PLAN-02	Każdy plan zawiera opis, poziom trudności i szacowany czas trwania.	Średni
FR-PLAN-03	Użytkownik może wybrać i aktywować wybrany plan treningowy.	Wysoki
FR-PLAN-04	Aktualnie wybrany plan jest wizualnie wyróżniony na liście.	Średni
FR-PLAN-05	System zapisuje wybrany plan treningowy w DataStore (persystencja między sesjami).	Wysoki
FR-PLAN-06	Ekran udostępnia filtry do przeszukiwania i zawężania listy planów.	Średni
FR-PLAN-07	Wybrane filtry są zachowywane po zamknięciu panelu filtrowania.	Średni
FR-PLAN-08	Plany zawierają informacje o grupach mięśniowych, których dotyczą.	Niski

- **Centrum Treningowe (WorkoutHubScreen)**

ID	Wymaganie	Priorytet
FR-WORK-01	Ekran wyświetla listę ćwiczeń z aktualnie wybranego planu treningowego.	Wysoki
FR-WORK-02	Każde ćwiczenie prezentuje nazwę, opis, liczbę powtórzeń lub czas trwania.	Wysoki
FR-WORK-03	Ćwiczenia posiadają wideo demonstracyjne lub animację pokazującą poprawne wykonanie.	Wysoki
FR-WORK-04	System udostępnia timer odliczający czas ćwiczenia z sygnałem audio/wizualnym po zakończeniu.	Średni
FR-WORK-05	Użytkownik może oznaczyć ćwiczenie jako ukończone.	Wysoki
FR-WORK-06	System śledzi dzienny postęp wykonanych ćwiczeń.	Średni
FR-WORK-07	Wideo ćwiczenia jest poprawnie wyświetlane po obrocie ekranu urządzenia.	Wysoki
FR-WORK-08	System obsługuje 30+ różnych ćwiczeń w bazie danych.	Średni

- **Profil użytkownika (ProfileScreen)**

ID	Wymaganie	Priorytet
FR-PROF-01	Ekran wyświetla dane zalogowanego użytkownika: imię, adres e-mail.	Wysoki
FR-PROF-02	Użytkownik może zmienić swoje imię poprzez panel edycji (Bottom Sheet).	Średni
FR-PROF-03	Użytkownik może zmienić adres e-mail poprzez panel edycji.	Średni
FR-PROF-04	Użytkownik może zmienić hasło; zmiana jest zapisywana w bazie danych Room.	Wysoki
FR-PROF-05	System waliduje dane wprowadzane w polach edycji profilu (puste pola, format e-mail).	Wysoki
FR-PROF-06	Użytkownik może dodać lub zmienić zdjęcie profilowe (awatar).	Niski
FR-PROF-07	System wyświetla stosowny placeholder gdy plik obrazu jest uszkodzony lub niedostępny.	Średni

FR-PROF-08	Panel edycji profilu (Bottom Sheet) zachowuje dane po obroceniu ekranu.	Średni
FR-PROF-09	Imię użytkownika jest poprawnie wyświetlane po przełączeniu konta.	Średni
FR-PROF-10	Pola edycji profilu są walidowane pod kątem SQL Injection.	Wysoki

• Ustawienia

ID	Wymaganie	Priorytet
FR-SETT-01	System udostępnia trzy tryby motywu: Jasny (Light), Ciemny (Dark) oraz Systemowy (Follow System).	Wysoki
FR-SETT-02	Wybrany motyw jest stosowany natychmiast po wyborze bez restartu aplikacji.	Średni
FR-SETT-03	Wybrany motyw jest zapisywany w DataStore i przywracany po ponownym uruchomieniu aplikacji.	Wysoki
FR-SETT-04	System udostępnia wybór języka interfejsu: Polski i Angielski (i18n).	Wysoki
FR-SETT-05	Zmiana języka jest stosowana w całej aplikacji bez restartu.	Wysoki
FR-SETT-06	Wybrany język jest zapisywany w DataStore i przywracany po restarcie.	Wysoki
FR-SETT-07	Zmiana języka systemu nie nadpisuje języka wybranego przez użytkownika w ustawieniach.	Średni
FR-SETT-08	Panel ustawień (Bottom Sheet) zachowuje stan po obroceniu ekranu.	Średni
FR-SETT-09	Na urządzeniach z API ≥ 33 zmiana języka korzysta z <code>AppCompatDelegate.setApplicationLocales()</code> .	Wysoki
FR-SETT-10	System umożliwia włączenie i wyłączenie powiadomień przypominających o treningu.	Średni
FR-SETT-11	Użytkownik może skonfigurować harmonogram powiadomień (czas, częstotliwość).	Średni
FR-SETT-12	Panel pomocy (Help Bottom Sheet) wyświetla się poprawnie i zachowuje stan po rotacji.	Niski

• Powiadomienia oraz nawigacja (inne)

ID	Wymaganie	Priorytet
FR-NOTI-01	System tworzy kanały powiadomień Android (NotificationChannel) przy uruchomieniu aplikacji.	Wysoki
FR-NOTI-02	Powiadomienia są harmonogramowane za pomocą WorkManager (background task scheduling).	Wysoki
FR-NOTI-03	Panel powiadomień (Notifications Bottom Sheet) zachowuje ustawienia po obroceniu ekranu.	Średni
FR-NOTI-04	Ekran powitalny (SplashScreen) wyświetla animację logo i pasek postępu przez ok. 3 sekundy.	Niski
FR-NOTI-05	Po zakończeniu SplashScreen aplikacja automatycznie przekierowuje użytkownika do LoginScreen.	Wysoki
FR-NOTI-06	Nawigacja między ekranami korzysta z Jetpack Navigation Compose z Single Activity.	Wysoki
FR-NOTI-07	Cofnięcie ze stosu nawigacji (back stack) zachowuje poprawny stan ekranów.	Średni

FR-NOTI-08	Dolna nawigacja (bottom navigation) jest widoczna na wszystkich głównych ekranach po zalogowaniu.	Średni
------------	---	--------

4. Technologie i architektura

- **Architektura**

MVVM – Separacja warstw i odpowiedzialności w aplikacji.

Single Activity + Jetpack Navigation – Jedna aktywność zawierająca główny komponent a przejścia między ekranami są zarządzane w sposób uporządkowany.

Unidirectional Data Flow – Jednokierunkowy przepływ danych oparty o StateFlow

- **Użyte technologie**

Kotlin – Główny język programowania

Jetpack Compose – Biblioteka do deklaratywnego tworzenia interfejsu użytkownika

Hilt – Wstrzykiwanie zależności (DI) do zapewnienia lepszej modułowości w plikach viewmodelu i przekazywaniu obiektów pomiędzy warstwami.

Room + SQLite – Lokalna baza danych

DataStore – przechowywanie danych konfiguracyjnych (key-value)

WorkManager – zadania w tle (np. system powiadomień)

Jetpack Navigation Compose – Nawigacja pomiędzy ekranami, zarządzanie flow aplikacji.

StateFlow / LiveData – zarządzanie stanem aplikacji

Coroutines – operacje asynchroniczne w aplikacji, wykonywanie operacji w tle, aby nie blokować

5. Plan testów

Głównym celem procesu testowania w przypadku tego projektu jest wykrycie wszystkich problemów z głównymi funkcjonalnościami wersji MVP, przećwiczeniu praktycznego aspektu testowania manualnego, podstaw automatyzacji testów UI oraz integracyjnych. Całość procesu testowego została przeniesiona do narzędzia Jira w celach praktycznego wykorzystania metodyki pracy opartej o sprinty i zarządzanie zadaniami.

6. Strategia testowania

- **Testy manualne**
- **Testy integracyjne** (JUnit, Mockito)
- **Testy integracyjne** (Room DB)
- **Testy UI/Espresso** (podstawy automatyzacji)
- **Testy bezpieczeństwa** (SQL Injection)
- **Testy i18n** (język PL/EN zastosowany w aplikacji)
- **Testy rekompozycji i persystencji danych**

7. Środowisko testowe

- Android Studio (emulator + device)
- Pixel 5 — API 33 (emulator)
- Medium Phone — API 36 (emulator)
- Xiaomi Mi Mix 2s (physical device)
- Jira (zarządzanie)
- Git / GitHub (wersjonowanie)
- ADB (debugowanie)

8. Scenariusz testów

- **Podział testów**

Aby odwzorować odpowiednie środowisko pracy w narzędziu Jira, w projekcie zastosowałem podział na epiki, story oraz odpowiednie sprinty.

Epiki

- **SITZERB4RE-9 – User Authentication Testing**

LoginScreen Manual Testing, RegisterScreen Manual Testing, LoginScreen Unit Testing, RegisterScreen Unit Testing.

Co pokrywa: logowanie, rejestracja, walidacja pól, SQL Injection, rekompozycja, nawigacja.

- **SITZERB4RE-7 – Workout Module Testing**

PlanSelectScreen Manual Testing, PlanSelectScreen Unit Testing, WorkoutHub Manal Testing, WorkoutHub Unit Testing.

Co pokrywa: filtry planów, wybór planu, odtworzenie wideo, timer, rekompozycja komponentów i zachowanie danych.

- **SITZERB4RE-8 – Profile & Settings Testing**

ProfileScreen Manual Testing, ProfileScreen Unit Testing.

Co pokrywa: Edycja profilu, zmiana hasła/email, avatar, motyw, język, powiadomienia, SQL Injection

- **SITZERB4RE-64– Home Testing**

HomeScreen Manual Testing, HomeScreen Unit Testing

Co pokrywa: onboarding, kategorie, widok planów, nawigacja, stan po rekompozycji i zachowanie danych.

- **SITZERB4RE-10 – Cross-module Testing**

Theme Manual / Unit Testing, Notification Unit Testing, Navigation Manual/Unit Testing, Onboarding Manual/Unit Testing

Co pokrywa: Testowanie przekrojowe pomiędzy ekranami, zachowanie danych i ustawień pomiędzy komponentami aplikacji.

- **SITZERB4RE-2 – Przygotowanie środowiska**

Co pokrywa: Konfiguracja emulatorów, danych testowych, stabilnego brancha do testowania oraz środowiska CI.

- **Przypadki testowe**

Wszystkie przypadki testowe są dostępne w pliku **jira_test-cases.xlsx** w folderze **docs** projektu.

9. Defekty i analiza błędów w modułach aplikacji

Po przeprowadzonym procesie testowania aplikacji zostały znalezione 19 defekty, które zaraz przedstawię w formie tabeli. Znalezione błędy idealnie w tym przypadku rzutują na zasadę Pareto w testowaniu, ponieważ w przybliżeniu 80% błędów znajduje się w 20% komponentów aplikacji, głównie systemu autoryzacji oraz funkcjonalności w profilu użytkownika.

Dla przykładu przedstawię kilka zgłoszonych błędów w systemie Jira oraz jakie kroki podjąłbym w celu naprawienia błędów:

- **Moduł autoryzacji logowanie/rejestracja**

Tytuł	[BUG-LOG01] Błąd tłumaczenia pola błędu w ekranie logowania i rejestracji
Opis	Błąd typu i18n, podczas błędnej rejestracji/logowania pomimo ustawionego języka angielskiego w aplikacji/systemie jest to sztywna wartość po polsku.
Kroki do wykonania i dane testowe	1. Otwórz aplikację 2. Poczekać na koniec wyświetlania Splashscreena 3. Wpisz wartość E-mail z danych testowych w pole e-mail 4. Wpisz hasło z danych testowych w pole hasło 5. Wciśnij przycisk logowania „Login” E-mail: jozefskowron91@gmail.com Password: jozefskowron123
Oczekiwany rezultat	Wyświetlona wiadomość błędu jest w języku aplikacji/systemu
Rzeczywisty rezultat	Wyświetlona wiadomość błędu jest w języku polskim niezależnie od wybranych ustawień językowych.

Tytuł	[BUG-REG01] Użytkownik może zarejestrować konto z podaniem e-mail w złym formacie
Opis	Błąd w walidacji pól e-mail, jeżeli wpisana wartość przypomina składnią prawdziwy e-mail to nie weryfikuje jego poprawności.
Kroki do wykonania i dane testowe	1. Otwórz aplikację 2. Poczekać na koniec wyświetlania Splashscreena

	3. Wciśnij podświetlony tekst „Sign In” 4. Wprowadź imię w pole imię z danych testowych 5. Wprowadź e-mail w pole e-mail z danych testowych 6. Wprowadź hasło w pole hasła z danych testowych 7. Powtórz hasło z danych testowych w polu potwierdzenia hasła 8. Kliknij przycisk „Sign In” E-mail: testmail!!^%@ppppp[Username: testuser123 Password: test123pass
Oczekiwany rezultat	Ekran rejestracji wyświetla odpowiednią wiadomość błędu, nowe konto nie zostało zarejestrowane.
Rzeczywisty rezultat	Użytkownik zarejestrował nowe konto z złym formatem e-mail, wiadomość błędu nie jest wyświetlona użytkownikowi.

Tytuł	[BUG-REG02] Użytkownik może zarejestrować konto z bardzo długim hasłem (>200 znaków)
Opis	Błąd walidacji hasła, można uznać, że istnieją zasady pewnego hasła ale jednak hasło składające się z więcej niż 200 znaków może być problematyczne do obsłużenia.
Kroki do wykonania i dane testowe	1. Otwórz aplikację 2. Poczekać na koniec wyświetlania Splascreena 3. Wciśnij podświetlony tekst „Sign In” 4. Wprowadź imię w pole imię z danych testowych 5. Wprowadź e-mail w pole e-mail z danych testowych 6. Wprowadź hasło w pole hasła z danych testowych 7. Powtórz hasło z danych testowych w polu potwierdzenia hasła 8. Kliknij przycisk „Sign In” E-mail: jozefskowron91@gmail.com Username: skowron91 Password: jooooozteteaetaetttttttooofzefskoozzzzzzzzzzzzzzttttttteteffffffffffffffffffffffffffffffffffff ffffffffffffffffffffffffffffffffffffteooooowron91passssssssssss% % % % % % % % % % sssword 12345678910101010!!!!!!
Oczekiwany rezultat	Ekran rejestracji wyświetla odpowiednią wiadomość błędu, nowe konto nie zostało zarejestrowane.
Rzeczywisty rezultat	Użytkownik zarejestrował nowe konto z bardzo długim hasłem, wiadomość błędu nie jest wyświetlona użytkownikowi.

Dla zgłoszonego błędu **BUG-LOG-01** o niskim priorytecie rozwiązanie będzie prostym zadaniem, po zbadaniu kodu można zauważyć, że wartość naszego komunikatu błędu jest ustawiana na sztywno w języku polskim w metodzie logowania/rejestracji

```
else {  
    errorMessage = "Zły email lub hasło"  
    loginSuccess = false  
}
```

Dla zgłoszeń **BUG-REG02** i **BUG-REG01** rozwiązanie zostało szybko przedstawione, zaglądając do sposobu walidacji rejestracji użytkownika, można zauważyć, że jedynym warunkiem sprawdzającym długość hasła jest, jeżeli hasło jest mniejsze niż 5 znaków, a format e-mailu nie jest poprawnie obsługiwany.

```
fun registerUser() { 11 Usages  
    //Podstawowa walidacja dla rejestracji  
    if (firstName.isEmpty() || email.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {  
        errorMessage = "Uzupełnij wszystkie pola!"  
        return  
    }  
  
    if (password != confirmPassword) {  
        errorMessage = "Hasła nie są identyczne!"  
        return  
    }  
  
    if (password.length < 5) {  
        errorMessage = "Hasło musi mieć co najmniej 5 znaków!"  
        return  
    }  
}
```

- **Moduł centrum ćwiczeń WorkoutHub**

Tytuł	[BUG-WOR01] Zmiana orientacji ekranu podczas wyświetlania karty z ćwiczeniem powoduje jej nieprawidłowe wyświetlanie
Opis	Zmiana orientacji ekranu podczas ćwiczeń powoduje jej błędne wyświetlanie, karta z prezentacją ćwiczenia oraz panelem nawigacyjnym chowa się, elementy są źle rozmieszczone.
Kroki do wykonania i dane testowe	1. Zaloguj się uzupełniając pola z danych testowych. 2. Wybierz „plans” z dolnego paska nawigacyjnego 3. Wybierz kategorię „core” z paska kategorii 4. Kliknij plan o nazwie „Core Crusher” z listy 5. Wciśnij przycisk „start workout” w komponencie bottom sheet. 6. Wciśnij przycisk „następny” w nawigacji planu dwa razy 7. Wciśnij przycisk „start” w nawigacji planu.

	8. Obróć urządzenie do pozycji poziomej. E-mail: jozefskowron91@gmail.com Password: jozefskowron123
Oczekiwany rezultat	Wideo z ćwiczenia „Crunches” jest poprawnie odtworzone, nazwa i opis ćwiczenia jest poprawnie rozmieszczona i wyświetlona użytkownikowi.
Rzeczywisty rezultat	Po zmianie orientacji ekranu, karta z wideo, opisem i nazwą ćwiczenia jest ukryta i zapadnięta.

Testy zachowania ekranów i wartości w polach podczas rekompozycji znalazły swoje zastosowania, gdyż ekran WorkoutHubScreen okazał się nie reagować odpowiednio na horyzontalną rotację ekranu. Winowajcą okazał się układ elementów który w pozycji poziomej jest odpowiednio rozmieszczony przy użyciu wartości wag **weight(1f)**, lecz przy horyzontalnym ekranie te wartości się mocno rozjeżdżają, powodując błędy w wyświetlaniu, rozwiązaniem więc będzie zastosowanie wartości **scrollable** do głównej kolumny elementów.

```
{ paddingValues ->
    val scrollState = rememberScrollState()
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(scrollState)
            .padding(paddingValues)
            .padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
```

- **Moduł profilu i ustawień**

Tytuł	[BUG-PRO02] Próba ustawienia awataru użytkownika uszkodzonym zdjęciem powoduje nieprawidłowe wyświetlanie placeholdera.
Opis	Plik graficzny wygenerowany przez stronę „Online corrupt file generator” po ustawieniu jako zdjęcie profilowe sprawia, że wyświetlany domyślnie awatar jest niepoprawny.
Kroki do wykonania i dane testowe	1. Jako zalogowany użytkownik kliknij „Profile” w dolnym pasku nawigacyjnym. 2. Kliknij ikonę długopisa „Edit” przy zdjęciu profilowym 3. Wybierz podświetlony test „select picture” 4. Wybierz wygenerowany zepsuty plik graficzny corrupted_img.jpg z galerii zdjęć. 5. Wciśnij przycisk „Select” 6. Wciśnij przycisk „Save” w komponencie bottom sheet. E-mail: jozefskowron91@gmail.com

	Password: jozefskowron123  corrupted_img(1).jpg g
Oczekiwany rezultat	Aplikacja powinna wyświetlić stosowną wiadomość błędu oraz ustawić zdjęcie profilowe jako wartość domyślną.
Rzeczywisty rezultat	Aplikacja nie wyświetla komunikatu błędu a zdjęcie profilowe jest ustawione jako puste miejsce zamiast domyślnej wartości.

Tytuł	[BUG-PRO09] Zapisane ustawienia języka w aplikacji zmieniają się po zmianie języka systemowego.
Opis	Zmiana wartości języka systemowego po wybraniu języka aplikacji sprawia, że po ponownym włączeniu aplikacji następuje samoistna zmiana języka aplikacji.
Kroki do wykonania i dane testowe	1. Przejdź do zakładki „Profile” w dolnym pasku nawigacji. 2. Otwórz „Settings”. 3. Wybierz język „Polski” z listy języków. 4. Kliknij przycisk „Save”. 5. Zamknij aplikację. 6. Zmień język systemowy urządzenia na angielski w ustawieniach telefonu. 7. Ponownie uruchom aplikację. 8. Po ekranie splash screen przejdź do ekranu logowania. E-mail: jozefskowron91@gmail.com Password: jozefskowron123
Oczekiwany rezultat	Język aplikacji powinien być ustawiony na wartość wybraną w ustawieniach czyli „Polski”, nie powinna nastąpić zmiana na język systemowy.
Rzeczywisty rezultat	Aplikacja samoistnie zmieniła wybrany w aplikacji język na język systemowy.

Zgłoszenie **BUG-PRO02** zostało naprawione poprzez odpowiednie dopasowanie parametrów do metody wywołanej podczas przetwarzania nowego awataru, biblioteka **Coil** umożliwia w łatwy sposób ustawienie pliku graficznego przy wczytywaniu nowego pliku graficznego oraz przy wystąpieniu błędu.

```
val avatarPainter = avatarUri?.let {
    rememberAsyncImagePainter(
        model = it,
        placeholder = painterResource(id = R.drawable.pfpp),
        error = painterResource(id = R.drawable.pfpp)
    )
} ?: painterResource(id = R.drawable.pfpp)
```